

1. En-tête et Constantes Physiques (Le cadre thermodynamique)

```
import math
```

- Importe la bibliothèque mathématique standard de Python pour utiliser les fonctions de calcul avancées comme les puissances (`math.pow`) et les exponentielles (`math.exp`).

```
G = 9.80665    # Pesanteur (m/s²)
M = 0.0289644  # Masse molaire de l'air (kg/mol)
R = 8.31445    # Constante des gaz parfaits (J/mol/K)
PO = 101325.0  # Pression au sol (Pa)
TO = 288.15    # Température au sol (K)
```

- Définit les constantes physiques universelles indispensables pour appliquer la loi des gaz parfaits et l'équation de l'hydrostatique dans l'atmosphère terrestre standard.

2. Base de Données Structurale (Le modèle de l'atmosphère)

```
ISA_LAYERS = [
    (11.0, 15.0, -6.5), # Troposphère
    (20.0, -56.5, 0.0), # Stratosphère basse
    (32.0, -56.5, 1.0), # Stratosphère moyenne
    (50.0, -44.5, 2.8), # Stratosphère haute
    (71.0, 5.9, -2.8),  # Mésosphère basse
    (85.0, -52.9, -2.0), # Mésosphère haute
    (100.0, -80.9, 12.0) # Thermosphère basse
]
```

- Stocke la structure de l'Atmosphère Normalisée Internationale (ISA) sous forme de liste de tuples (altitude_maximale, température_à_la_base, gradient_thermique) afin de modéliser les changements de comportement physique selon l'altitude.

```
GAS_REGIME = {
    "CO2": 425.0,
    "CH4": 1.91,
    "N2O": 0.33
}
```

- Centralisé sous forme de dictionnaire les concentrations de référence (en ppm) des gaz à effet de serre uniformément répartis dans la basse atmosphère (homosphère).

3. Fonction : Calcul de la Température Locale

```
def get_temperature_isa      float
```

- Déclare la fonction chargée de déterminer la température absolue (en Kelvin) à n'importe quelle altitude `z_km` donnée en entrée.

```
z_base = 0.0
```

```
for z_max, t_base_c, alpha in ISA_LAYERS:
```

```
    if z_km <= z_max:
```

```
        return (t_base_c + alpha * (z_km - z_base)) + 273.15
```

```
    z_base = z_max
```

- Parcourt les couches de l'atmosphère pour identifier celle où se trouve l'altitude cible, applique la formule d'évolution linéaire du gradient thermique ($T = T_{base} + \alpha \times \Delta z$), puis convertit le résultat du degré Celsius vers le Kelvin.

```
return (ISA_LAYERS[-1][1] + ISA_LAYERS[-1][2] * (z_km - z_base)) + 273.15
```

- Sécurité algorithmique qui calcule la température par extrapolation linéaire si l'altitude demandée dépasse la limite supérieure de la dernière couche définie.

4. Fonction : Calcul de la Pression (Intégration Hydrostatique)

```
def get_pressure_isa      float
```

- Déclare la fonction chargée de calculer la pression atmosphérique locale (en Pascal) à l'altitude `z_km` demandée.

```
p_current = P0
```

```
z_base = 0.0
```

- Initialise les variables de calcul en partant de la pression standard de référence mesurée au sol.

```
for z_max, t_base_c, alpha in ISA_LAYERS:
```

```
    z_end = min(z_km, z_max)
```

```
    dz_m = (z_end - z_base) * 1000.0
```

- Découpe l'altitude totale en segments correspondant aux couches atmosphériques traversées et convertit l'épaisseur de kilomètres en mètres pour respecter les unités du système international.

```
t_start_k = t_base_c + 273.15
```

```
alpha_m = alpha / 1000.0
```

- Convertit les données thermiques de la couche active (température de base en Kelvin et gradient thermique en Kelvin par mètre) pour s'aligner sur les unités des constantes physiques R et G .

```
if alpha_m != 0.0:
```

```
    t_final_k = t_start_k + alpha_m * dz_m
```

```
    p_current *= math.pow(t_final_k / t_start_k, (-G * M) / (R * alpha_m))
```

- Applique l'intégration de la loi hydrostatique combinée aux gaz parfaits pour les zones où la température varie (ex: troposphère), traduisant la décroissance de la pression sous forme de loi de puissance.

```
else:
```

```
    p_current *= math.exp((-G * M * dz_m) / (R * t_start_k))
```

- Applique la loi de Laplace (décroissance exponentielle pure de la pression) spécifique aux zones isothermes de l'atmosphère où la température reste constante (ex: basse stratosphère).

```
if z_km <= z_max:
```

```
    break
```

```
    z_base = z_max
```

```
return p_current
```

- Interrompt immédiatement l'intégration par paliers dès que l'altitude cible est atteinte, puis renvoie la valeur finale de la pression atmosphérique locale.

5. Fonction Polyvalente : Concentration des Gaz

```
def get_gas_concentration(float str
```

- Spécifie la fonction principale du projet chargée d'évaluer les concentrations du gaz choisi à l'altitude voulue.

```
gas = gas_name.upper()
```

```
t_k = get_temperature_isa(z_km)
```

```
p_pa = get_pressure_isa(z_km)
```

- Standardise le nom du gaz en majuscules pour éviter les erreurs de syntaxe, puis appelle les fonctions physiques pour obtenir l'état exact de l'air à cette altitude.

```
if gas in GAS_REGIME:
    ppm_melange = GAS_REGIME[gas]
```

- Assigne la proportion relative constante du gaz s'il fait partie des espèces chimiques uniformément brassées dans l'homosphère (CO₂, CH₄, N₂O).

```
elif gas == "O3":
    if 15.0 <= z_km <= 35.0:
        ppm_melange = 8.0 * math.exp(-((z_km - 25.0) / 5.0)**2)
    else:
        ppm_melange = 0.1
```

- Implémente une fonction mathématique en cloche (courbe gaussienne) pour simuler le profil vertical non-uniforme de la couche d'ozone (O₃), caractérisé par un pic de concentration dans la stratosphère.

```
else:
    raise ValueError("Gaz non pris en compte. Choisissez : CO2, CH4, N2O ou O3.")
```

- Sécurité algorithmique (gestion des erreurs) qui interrompt le programme et avertit l'utilisateur si le nom du gaz entré n'est pas pris en charge.

```
ppm_vol_abs = ppm_melange * (p_pa / P0) * (T0 / t_k)
```

- Utilise l'équation d'état des gaz parfaits pour corriger la concentration relative et obtenir la quantité réelle de molécules par unité de volume (PPM Volumique Absolu), qui diminue avec la raréfaction de l'air.

```
return {
    "Altitude": z_km,
    "Gaz": gas,
    "Temp_C": round(t_k - 273.15, 2),
    "Pression_Pa": round(p_pa, 1),
    "PPM_Melange": ppm_melange,
    "PPM_Vol_Abs": round(ppm_vol_abs, 3)
}
```

- Renvoie l'ensemble des grandeurs calculées sous la forme d'un dictionnaire structuré, facilitant l'exploitation future des données pour les affichages ou les tracés graphiques.

6. Bloc d'Exécution Principal (L'espace de simulation)

```
if __name__ == "__main__":
```

- Clause standard en Python qui garantit que le code de démonstration ci-dessous s'exécutera uniquement si ce fichier est lancé directement, préservant la réutilisation future du module par d'autres scripts.

```
print("--- VÉRIFICATION DU MODÈLE ATMOSPHERIQUE ---")
scenarios = [(0.0, "CO2"), (11.0, "CH4"), (25.0, "O3")]
```

- Initialise un jeu de données de test contenant des couples (altitude, gaz) stratégiques pour valider les équations et le comportement des différents profils de gaz.

```
for alt, g in scenarios:
    res = get_gas_concentration(alt, g)
    print(f"[{res['Gaz']}] à {res['Altitude']} km -> Temp: {res['Temp_C']}°C | P: {res['Pression_Pa']} Pa | PPM Mix: {res['PPM_Melange']} | PPM Vol Abs: {res['PPM_Vol_Abs']}")
```

- Parcourt chaque scénario de test, exécute l'algorithme complet, et met en forme une chaîne de caractères claire pour afficher les résultats numériques de manière lisible dans le terminal.
-